One effective alternative to translate this into code, in object-oriented programming, is to use inheritance, defining a common abstract class (a class that can’t be instantiated to an actual object) or an interface and two derived classes for leaves and internal nodes, that shares a common data and behavior (defined in the base, abstract class), but are implemented differently. Listing 10.2 shows a possible pseudo-code description of this pattern.

Listing 10.2 Alternative Implementation for SsNode: SsNodeOO

```
abstract class SsNodeOO
  #type tuple(k)
  centroid
  #type float
  radius

class SsInnerNode: SsNodeOO
  #type SsNode[]
  children
  function SsInnerNode(children=[])

class SsLeaf: SsNodeOO
  #type tuple(k)[]
  points
  function SsLeaf(points=[])
```

Although the implementation using inheritance might result in some code duplication and a greater effort to understand the code, it arguably provides a cleaner solution, removing the logic to choose the type of node that would otherwise be needed in each method of the class.